

1. Considere a função **tiraPrefixo :: String -> String -> Maybe String** que, caso a primeira string seja prefixo da segunda, retira-lhe esse prefixo.
Por exemplo, **tiraPrefixo “mal” “malcriado” == Just “criado”** e **tiraPrefixo “mal” “amalgama” = Nothing**

tiraPrefixo :: String -> String -> Maybe String

tiraPrefixo [] s = just s

tiraPrefixo (x:xs) [] = Nothing

tiraPrefixo (x:xs) (y:ys) = XXXXXXXX

Qual a expressão que deve aparecer em XXXXXXXX?

- A) if x == y then tiraPrefixo xs ys else Nothing
 - B) if x == y then Just (tiraPrefixo xs ys) else Nothing
 - C) if x == y then Just x : (tiraPrefixo xs ys) else Nothing
 - D) if x /= y then Nothing else Just (x:(tiraPrefixo xs ys))
2. A função pré definida **nub** remove as repetições de uma lista.

nub [] = []

nub (h:t) | XXXXXXXX = nub t

| otherwise = h:nub t

Qual a expressão que deve aparecer em XXXXXXXX?

- A) h == t
- B) h == []
- C) elem h t
- D) t == []

3. Considere a seguinte definição da função **insert** que insere um elemento numa lista ordenada:

```
insert x [] = [x]
insert x (h:t) | x <= h = x:h:t
                | x > h  = XXXXX
```

Qual a expressão que deve aparecer em XXXXX?

- A) x:insert h t
- B) h ++ [x:t]
- C) h:insert x t
- D) h:x:t
- E) x:t

4. Considere a definição abaixo da função **bub** sobre listas

```
bub (x:y:ys) | x <= z      = x:z:zs
              | otherwise = z:x:zs
              where (z:zs) = bub (y:ys)
bub 1 = 1
```

Indique qual o resultado correto de **bub** [5,3,1,2,4]?

- A) 1
- B) [1,2,3,4,5]
- C) 5
- D) [3,5,2,4,1]
- E) [1,5,3,2,4]

5. A função **delete :: Eq a => a -> [a] -> [a]** retorna a lista resultante de remover (a primeira ocorrência de) um dado elemento de uma lista, e se não existir nenhuma ocorrência, devolve a própria lista.

Selecione a implementação correta desta função

- A) `delete 0 x = 0`
`delete x (h:t) = if x == h then delete h (x:t) else delete x (h:t)`
- B) `delete _ [] = []`
`delete x (h:t) = if x == h then delete x t else h : delete x t`
- C) `delete _ [] = []`
`delete x (h:t) | mod x 2 == 0 = h : delete x t`
`| otherwise = delete x t`
- D) `delete _ [] = []`
`delete x (h:t) | x == h = t`
`| otherwise = h : delete x t`
- E) `delete _ x = x`
`delete x xs = if x == xs then x : delete x xs else xs : delete x xs`

6. A função **triplos :: [a] -> [(a,a,a)]** recebe uma lista e vai construindo triplos com os elementos dessa lista (seguindo a mesma ordem em que elementos estão na lista). Quando já não existirem elementos suficientes para formar triplos, esses elementos são desprezados.

Por exemplo: **triplos "abcdefghijl" == [(‘a’, ‘b’, ‘c’), [(‘d’, ‘e’, ‘f’), [(‘g’, ‘h’, ‘i’)]]**

Selecione a implementação correta desta função:

- A) `triplos (a:b:c:t) = (a,b,c) : triplos t`
`triplos (a:b:[]) = [(a,b)]`
`triplos _ = []`
- B) `triplos ((a:b:c) : t) = (a,b,c) : triplos t`
`triplos _ = []`
- C) `triplos (a:b:c:t) = (a,b,c) : triplos t`
`triplos _ = []`
- D) `triplos _ = []`
`triplos (a:b:c:t) = (a,b,c) : triplos t`

7. Considere a função **descomprime** :: [(a, Int)] -> [a] que replica cada elemento da lista de entrada ou número de vezes especificado.

Exemplo: **descomprime** [(‘a’, 2), (‘b’, 4)] == “aabbbb”

```
descomprime :: [(a, Int)] -> [a]
descomprime [] = []
descomprime (h:t) = XXXXX
```

```
fun :: (a, Int) -> [a]
fun (x, 0) = []
fun (x, n) | n > 0 = x : fun (x, n-1)
```

Qual a expressão que deve aparecer em XXXXX?

- A) (fun h) ++ (descomprime t)
- B) (fst h) : (descomprime t)
- C) h : (descomprime t)
- D) (fun h) : (descomprime t)

8. Considere a seguinte função **union**:

```
union :: Eq a => [a] -> [a] -> [a]
union l [] = l
union l (x:xs)
  | elem x l = union l xs
  | otherwise = x : union l xs
```

Indique qual o resultado correto de **union** [2,3,6] [2,1,3,1,6,7]

- A) [2,3,6,1,7]
- B) [1,1,7,2,3,6]
- C) [2,3,6,7,1,1]
- D) [7,1,2,3,6]

CHAVE : ACCEDCAB